What is SSRF DEFENSE in guardrails?

SSRF defense in guardrails means controls that stop an AI agent, tool, or web service from being tricked into making requests to internal or unintended destinations. SSRF stands for **Server-Side Request Forgery**: an attacker supplies a URL or host, and the server makes a network request on their behalf. [1] [2]

Why it matters

If an LLM agent has a "fetch this URL" or web-browsing tool, a malicious prompt or input could direct it to `http://localhost`, a cloud metadata endpoint, or an internal service. The agent then becomes an unintended proxy into the private network, potentially exposing credentials, configuration, or internal APIs. [3] [4]

What an SSRF guardrail typically does

- **Allows only approved destinations:** Uses an allowlist of domains, hosts, or URL patterns rather than trying to block known bad URLs. [4] [5]
- **Blocks private and reserved addresses:** Rejects `localhost`, `127.0.0.0/8`, private ranges such as `10.0.0.0/8`, `172.16.0.0/12`, `192.168.0.0/16`, and link-local `169.254.0.0/16`. [3]
- **Validates after DNS resolution:** Resolves the hostname and checks the actual IP address, helping prevent tricks such as DNS rebinding. [5] [3]
- **Restricts schemes and ports:** Usually permits only `https://` and only the ports required by the integration. [1] [5]
- **Handles redirects safely:** Does not automatically follow redirects, or revalidates every redirect target. [5] [1]
- **Limits blast radius:** Uses outbound-network restrictions, short timeouts, least-privilege service identity, and logging/monitoring. [1] [5]

Example

A guardrail might allow:

```
https://api.example.com/...
```

but block:

```
http://169.254.169.254/latest/meta-data/
http://localhost:8080/admin
```

```
https://internal-service.corp.local/
```

The first blocked URL is especially important in cloud environments because instance-metadata services can expose temporary cloud credentials if reachable through SSRF.[3]

In short, **SSRF defense is a safety control for any agent/tool that can make HTTP requests**: it ensures the request goes only to explicitly trusted public destinations, not to the server's own network or sensitive internal services.[4] [5]

⁂

1. https://developer.mozilla.org/en-US/docs/Web/Security/Attacks/SSRF
2. https://cheatsheetseries.owasp.org/cheatsheets/Server_Side_Request_Forgery_Prevention_Cheat_Sheet.html
3. https://builder.aws.com/content/3J70gFj7tGbI7iZx4Mcdx7NRIJm/four-guardrails-for-an-llm-that-accepts-anonymous-uploads
4. https://www.stackai.com/insights/how-to-design-ai-agent-guardrails-best-practices-for-input-validation-output-filtering-and-safety-controls
5. https://cornucopia.owasp.org/taxonomy/OWASP-top-10/10-server-side-reguest-forgery
6. https://github.com/cudocharles/OWASP-CheatSheetSeries/blob/master/cheatsheets/Server_Side_Request_Forgery_Prevention_Cheat_Sheet.md
7. https://docs.guardrails.io/docs/vulnerability-classes/insecure-processing-of-data/ssrf
8. https://github.com/at0m-b0mb/PurpleMCP
9. https://github.com/at0m-b0mb/PurpleMCP/blob/94fc1ab/attacks/05_ssrf/exploit.py
10. https://docs.definable.ai/agents/security